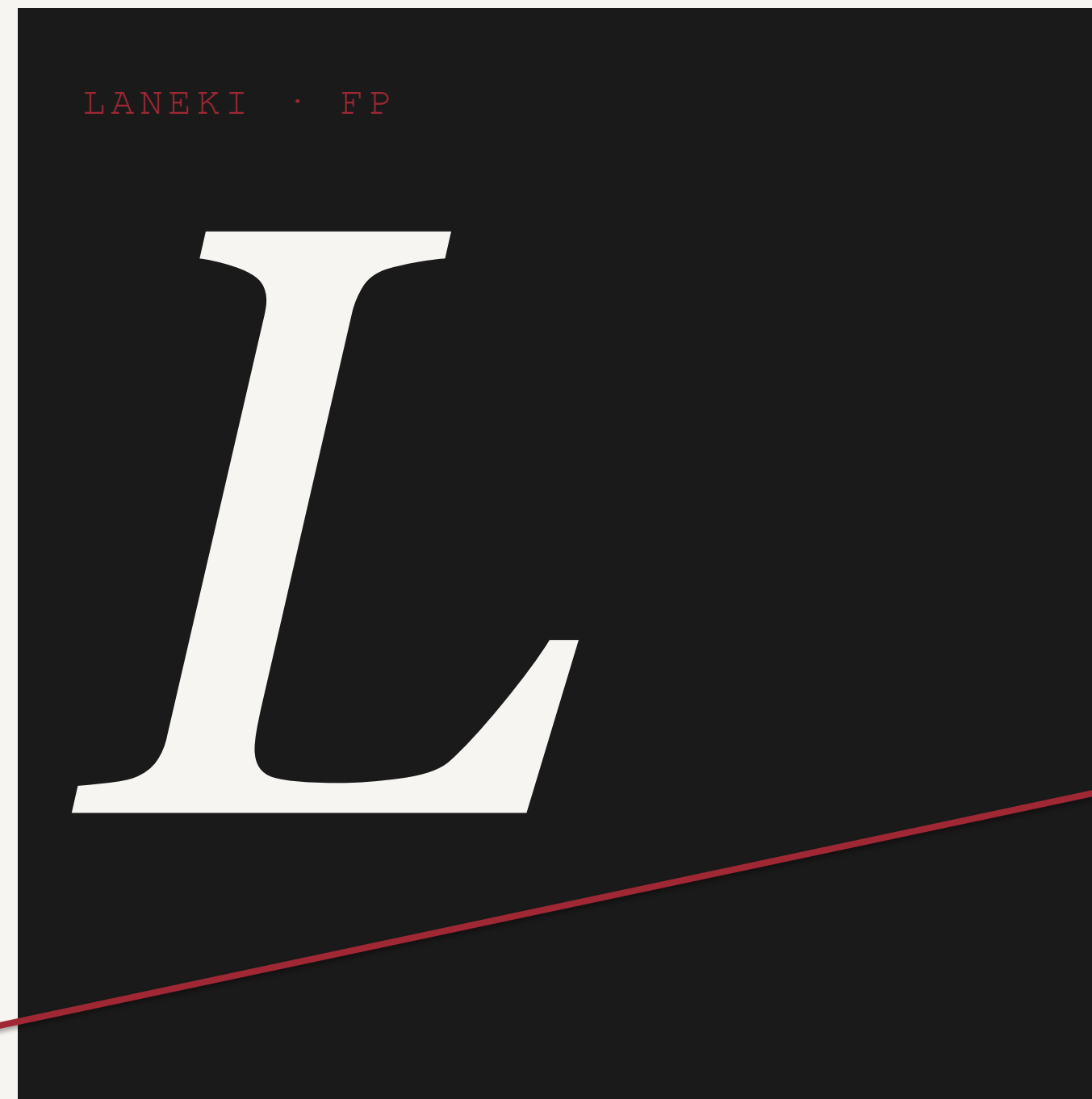


Apache *Kafka*

Denbora errealeko gertaera-streaming plataforma: topikak, brokerrak, kontsumitzaile-taldeak eta Python integrazioa.

Zikloa Adimen Artifiziala eta Big Data

Modulua Big Data Aplikatua



01 DATAFLOW. Apache Kafka

01 Sarrera — zer da Kafka, zergatik eta noiz

02 Argitaratu-Harpidetu eredua (Pub/Sub)

03 O. kasua — Kafka kaixo (zerbitzuak + komando-lerroa)

04 Elementuak — topic-ak, partizioak, brokerrak, errepikatze

05 Ekoizleak — ACK konfigurazioa, mezu-gakoak

06 Kontsumitzaileak eta kontsumitzaile-taldeak

07 Broker Discovery eta Zookeeper-en rola

08 1. kasua — Kafka Python-ekin integratuta

09 Hiru ariketa praktiko — DF3.1, DF3.2, DF3.3

01 Sarrera

Apache Kafka

Apache Kafka

Denbora errealeko GERTAEREN STREAMINGA egiteko plataforma BANATU eta KODE IREKIKOA da. Gertaerak modu IRAUNKORREAN gordetzeko eta sistema desberdinen artean banatzeko aukera ematen du.

MIDDLEWARE gisa funtzionatzen du, EKOIZLEAK (Producers) eta KONTSUMITZAILEAK (Consumers) elkarrengandik bereiziz.

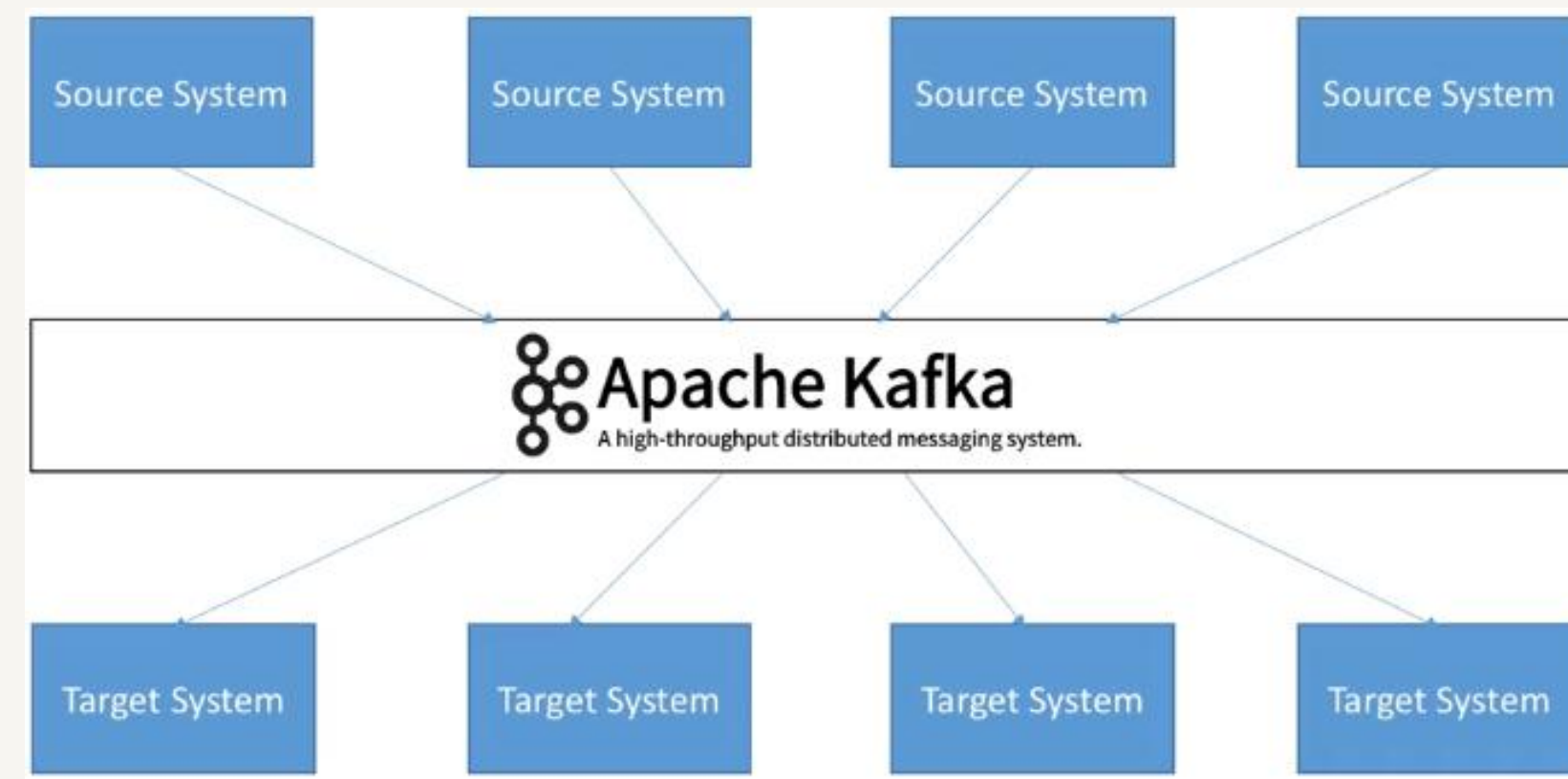
MIDDLEWARE-A

Ekoizleak eta kontsumitzaileak ELKARRENGANDIK ASKEAK dira: ekoizleak ez du jakin behar nork kontsumituko dituen bere datuak; kontsumitzaileak bere erritmoan iraku ditzake Kafka-n oraindik gordeta dauden gertaerak.

— MIDDLEWARE-A

Kafka middleware gisa — sistemen arteko zubi

- **GOIKO ALDEAN: ITURRI-SISTEMAK** (web aplikazioak, mugikorreko aplikazioak, IoT gailuak, mikrozerbitzuak...)
- **BEHEKO ALDEAN: KONTSUMITZAILE-SISTEMAK** (Data Warehouse, analitika, Machine Learning, auditoria...)
- **ERDIAN: KAFKA** — iturriak eta kontsumitzaileak Kafka-ra konektatzen dira
- **KAFKA GABE:** Producer eta Consumer-en arteko integrazio zuzen ugari (N x M arte)
- **KAFKA-REKIN:** sistema bakoitza Kafka-ra konektatzen da (N+M integrazio): arkitektura sinpleagoa eta mantentzeko errazagoa



Ezaugarri nagusiak eta erabilera-kasuak

EZAUGARRI NAGUSIAK:

- **THROUGHPUT HANDIA ETA LATENTZIA BAXUA** — gertaera-fluxu oso handiak denbora errealean kudeatzeko diseinatua
- **AKATS-TOLERANTZIA + ESKALAGARRITASUN HORIZONTALA + ERABILGARRITASUN HANDIA**
- **ESKALA HANDIKO PRODUKZIO-INGURUNEETAN OSO ERABILIA** — LinkedIn, Uber, Netflix, Spotify...

ERABILERA KASUAK:

- **MEZULARITZA ETA DATU-INGESTIOA** — aplikazio, log, clickstream eta bestelako gertaerak jasotzea
- **STREAMING PROZESAMENDUA ETA DENBORA ERREALEKO ANALITIKA**
- **MIKROZERBITZUAK** — event-driven komunikazioa
- **IoT** — sentsoire eta gailuetatik datozen gertaera-fluxuak kudeatzea

02

Argitaratu- Harpidetu eredua

Argitaratu-Harpidetu (Pub/Sub)

Pub/Sub eredua

Hiru elementu nagusi ditu:

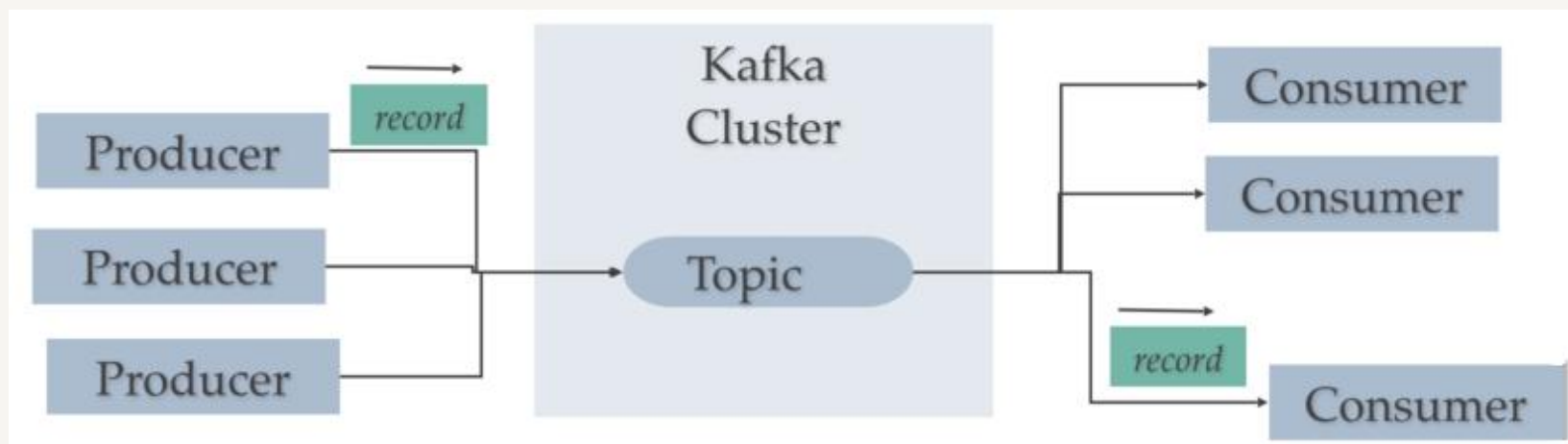
- **EKOIZLEA (Producer)**: gertaerak/mezuak sortu eta **topic** batera bidaltzen ditu.
- **TOPIC-A (Topic)**: mota bereko gertaerak/mezuak antolatu eta gordetzen dituen **kanal** logikoa da.
- **KONTSUMITZAILEA (Consumer)**: topic bateko gertaerak/mezuak irakurtzen ditu.

BEREIZIAK

Producer-a EZ da zuzenean Consumer-arekin komunikatzen: dena Topic-en bidez egiten da. BEREIZITA DAUDE: Producer-ak Topic-a ezagutzen du eta Consumer-ak Topic-a ezagutzen du, baina ez dute elkar ezagutu behar. Pub/Sub ereduaren abantaila nagusia: DESAKOPLAMENDUA.

— PUB/SUB ESKEMA

Producer → Topic → Consumer fluxu basikoa



- **PRODUCER-AK** mezua **TOPIC** batera bidaltzen du (send)
- **KAFKA-K** mezua topic-ean gordetzen du
- **CONSUMER-AK** topic-etik mezuak irakurtzen ditu (poll), bere erritmoan
- Topic berean **hainbat Producer eta Consumer** egon daitezke.
- **Mezua ez da ezabatzen irakurtzean** — retention-denboran topic-ean gordeta jarraitzen du

03

0. Kasua — Kafka Kaixo

0. KASUA – Kafka Kaixo

- **HELBURUA:** Kafka Docker-en martxan jarri, iabd-topic izeneko topic bat sortu, Producer batetik mezuak bidali eta Consumer batean jasotzen direla ikusi.
- Amaieran hau izango dugu:

Producer → iabd-topic → Consumer

— 0. KASUA: KAFKA KAIXO

1. Kafka Docker bidez abiarazi

1. Sortu ariketaren karpeta:
C:\kafka\ariketak\0-kasua-kafka-kaixo
2. Sortu **compose.yaml** fitxategia karpeta horretan:

```
C: > kafka > ariketak > 0-kasua-kafka-kaixo > compose.yaml
1  services:
2    kafka:
3      image: apache/kafka:latest
4      container_name: iabd-kafka
5      ports:
6      - "9092:9092"
```

3. Abiarazi Kafka:
docker compose up -d
4. Egiaztatu container-a martxan dagoela (espero den container-a: *iabd-kafka*)
docker ps

2. Topic-a sortu

1. Sartu Kafka container-ean:

```
docker exec -it iabd-kafka bash
```

2. Sortu *iabd-topic*:

```
/opt/kafka/bin/kafka-topics.sh --create --topic iabd-topic --bootstrap-server  
localhost:9092
```

3. Topic-ak zerrendatu:

```
/opt/kafka/bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

4. Topic-a deskribatu:

```
/opt/kafka/bin/kafka-topics.sh --describe --topic iabd-topic --bootstrap-server  
localhost:9092
```

3. Producer-a – Mezuak bidali

Ireki beste PowerShell terminal bat:

1. Sartu Kafka container-ean:

`docker exec -it iabd-kafka bash`

2. Abiarazi Producer-a:

`/opt/kafka/bin/kafka-console-producer.sh --topic iabd-topic --bootstrap-server localhost:9092`

3. Bidali mezu batzuk (lerro bakoitza mezu bat da):

> **Kaixo Kafka**

> **Hau nire lehen mezua da**

> **Big Data Aplikatua**

4. Consumer-a – Mezuak irakurri

Ireki beste PowerShell terminal bat:

1. Sartu Kafka container-ean:

```
docker exec -it iabd-kafka bash
```

2. Abiarazi Producer-a:

```
/opt/kafka/bin/kafka-console-consumer.sh --topic iabd-topic --from-beginning --  
bootstrap-server localhost:9092
```

Emitza:

Kaixo Kafka

Hau nire lehen mezua da

Big Data Aplikatua

5. Streaming-a probatu

Utzi **Producer** eta **Consumer** terminalak biak martxan.

1. Producer-ean bidali mezu batzuk (lerro bakoitza mezu bat da):

- > Laugarren mezua
- > Bosgarren mezua

Zer ikusi dugu?

- Producer-ak mezuak **Topic-era bidaltzen ditu**
- Kafka-k mezuak **gordetzen ditu**
- Consumer-ak mezuak **irakurtzen ditu**
- Producer eta Consumer-ek **aldi berean** lan egin dezakete
- Consumer-ak mezu bat irakurtzeak **EZ du mezua ezabatzen**

6. *--from-beginning* probatu eta amaitu

1. Gelditu Consumer-a: **Ctrl + C**
2. Berrirori abiarazi Consumer-a:
/opt/kafka/bin/kafka-console-consumer.sh --topic iabd-topic --from-beginning --bootstrap-server localhost:9092

Zer gertatzen da?

Aurreko mezuak **berrirori agertzen dira**

Beraz: **IRAKURRI ≠ EZABATU**

Kafka-k mezuak Topic-ean gordetzen jarraitzen du **retention-politikaren arabera**

3. Amaitzeko: **Ctrl + C**
4. Container-etik atera: **exit**
5. Kafka gelditu:
docker compose down

04

Kafka-ren barne-arkitektura

Topic-a eta Partizioa

Topic + Partition

TOPIC-A datu-korrante jakin bat da, ILARA gisa funtzionatzen duena. Topic-ak PARTIZIOTAN banatzen dira (0tik zenbakituta). Partizio bakoitzak mezuak ORDENATUTAKO COMMIT-LOG moduan gordetzen ditu, mezu bakoitzak OFFSET deritzon identifikatzaile gehikorra duelarik.

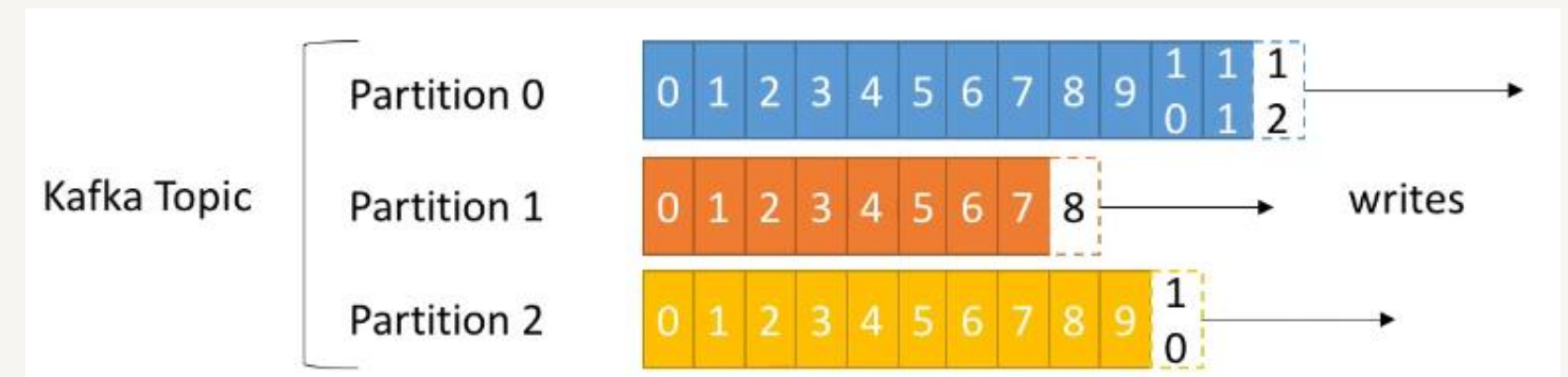
PARALELISMOA

PARTIZIOA = paralelismoaren UNITATEA. N partizio = N kontsumitzaile aldi berean irakurri ditzakete topic bat. Partizio-kopurua MAX-PARALELISMOA da.

— PARTIZIOAK

Topic-a N partiziotan — ordena BARRUAN bermatua

- TOPIC bakar bat (adib. 'iabd-topic') = N partizio (0, 1, 2...)
- PARTIZIO BAKOITZA = ordenatutako COMMIT-LOG bat (append-only diskoan)
- MEZU bakoitza partizio batera doa (gakoak edo round-robin-ek erabakitzen)
- **ORDENA PARTIZIOAREN BARRUAN bermatuta** — PARTIZIOEN ARTEAN EZ
- OFFSET = mezuaren KOKAPENA partizioaren barruan (zero-indexed)



Topic-en eta partizioen ezaugarri nagusiak

- **OFFSET partizio-MAILAN da** — ez globala (partizio bakoitzak bere kontagailua)
- **ORDENA PARTIZIO BARRUAN** baino ez da bermatzen (ez topic osoan)
- **RETENTION lehenetsia: ASTEBETE** — gero ezabatuak (konfiguragarria, 7 egun → infinite)
- **MEZUAK ALDAEZINAK behin idatzita** — append-only log eredua
- **PARTIZIO-ESLEIPENA: lehenetsiz AUSAZKOA** (round-robin); **GAKO-BIDEZ DETERMINISTIKOA**

Topic berriak partizioekin + ezabatzea

●●● topic-partizioak.sh

bash

```
1 # 1) 3 partiziozko topic-a sortu
2 kafka-topics.sh --create --topic iabd-topic-p3 \
3   --partitions 3 \
4   --bootstrap-server iabd-virtualbox:9092
5
6 # 2) Topic-a deskribatu (3 partizio agertu behar)
7 kafka-topics.sh --describe --topic iabd-topic-p3 \
8   --bootstrap-server iabd-virtualbox:9092
9
10 # 3) Topic-a EZABATU
11 kafka-topics.sh --delete --topic iabd-topic \
12   --bootstrap-server iabd-virtualbox:9092
13
14 # OHARRAK:
15 #   --partitions handitu daiteke gerora (--alter)
16 #   --partitions JEITSI ezin da (data loss arriskua)
17 #   --delete: delete.topic.enable=true behar broker-an
18 #   (default true Kafka berrietan)
```

Broker

Broker

Kafka kluster bat BROKER deritzon hainbat nodotan osatuta dago, bakoitza OSOKO ID batekin identifikatua. Broker bakoitzak PARTIZIOEN AZPIMULTZO bat gordetzen du (inoiz ez datu osoak, izaera banatuagatik). Edozein BOOTSTRAP broker-era konektatzeak kluster osora SARBIDEA ematen du.

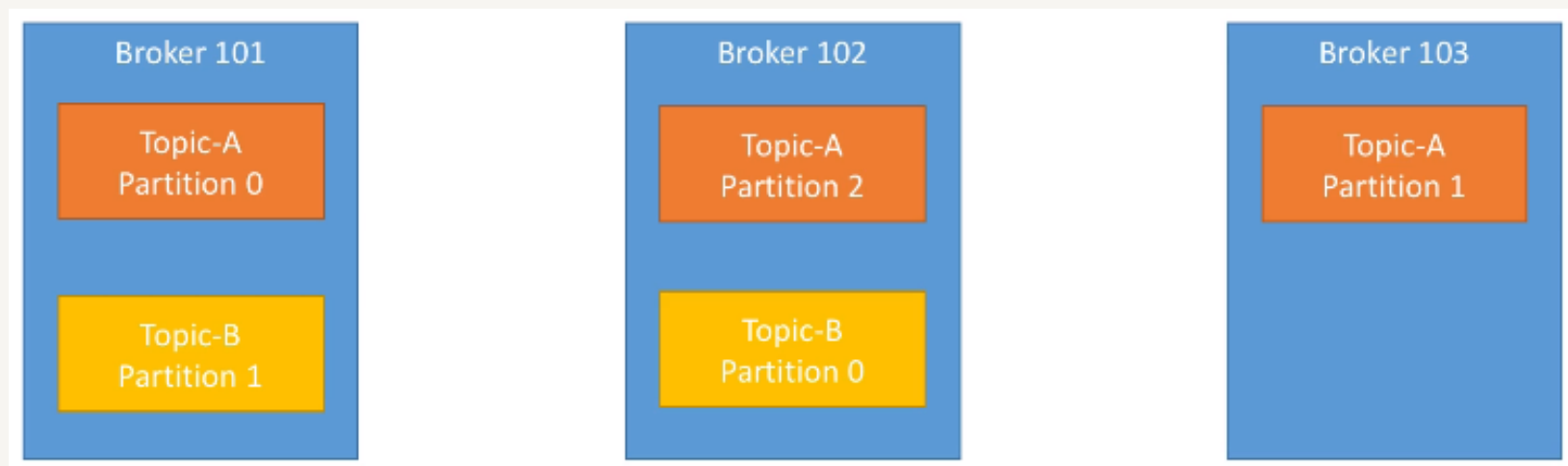
BANATUA ESKALAGARRI

Broker bat = zerbitzari bat. Datuak PARTIZIOZ partizio banatzen dira broker-en artean, eta erreplikatuak. Broker erortzeak EZ DU datu-galera sortzen (errepikak daudelako).

— KLUSTER BANATUA

3 broker-eko Kafka kluster adibidea

- **Broker 1, Broker 2, Broker 3** — bakoitza ZERBITZARI INDEPENDENTEA
- **Partizioak BROKER-EN ARTEAN BANATUTA** — bakoitzak azpimultzo bat
- **BOOTSTRAP edozein broker-i konektatuz** — kluster osoaren metadatu
- Broker bat erortzeak EZ du zerbitzua interruptitzen (errepiken erabilera)
- **PRODUKZIO-MINIMUM: 3 broker** (high availability + majority quorum)



04b

Errepikatze- faktorea

Errepikatze-faktoredun topic-a sortu

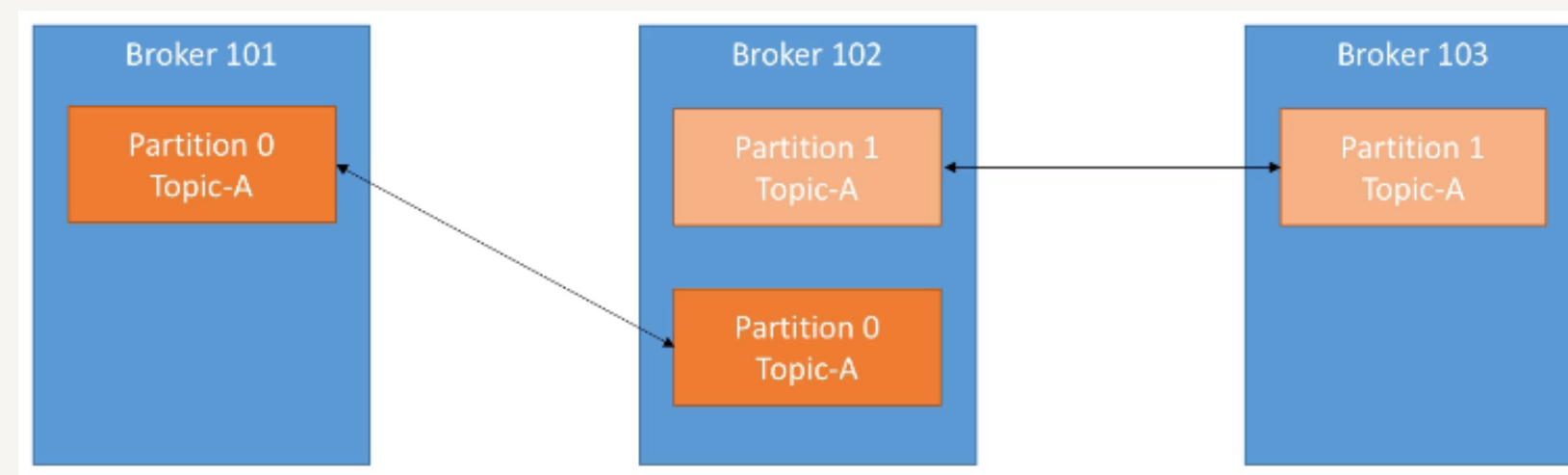
```
●●● topic-replikatua.sh bash

1 # 2 partizio + 2 errepikatze-faktore duen topic-a
2 kafka-topics.sh --create --topic TopicA \
3   --partitions 2 \
4   --replication-factor 2 \
5   --bootstrap-server iabd-virtualbox:9092
6
7 # DESCRIBE EMAITZA-ADIBIDEA:
8 #   Topic: TopicA  PartitionCount: 2  ReplicationFactor: 2
9 #   Topic: TopicA  Partition: 0  Leader: 0  Replicas: 0,1  Isr: 0,1
10 #   Topic: TopicA  Partition: 1  Leader: 1  Replicas: 1,0  Isr: 1,0
11 #
12 # INTERPRETAZIOA:
13 #   Partition 0 -> Leader 0, kopia 0 eta 1 broker-etan
14 #   Partition 1 -> Leader 1, kopia 1 eta 0 broker-etan
15 #   Kafka-k LIDER-AK BANATZEN ditu broker-en artean
16
17 # ARAUA: replication-factor <= broker-kopurua
```

— REPLIKAK

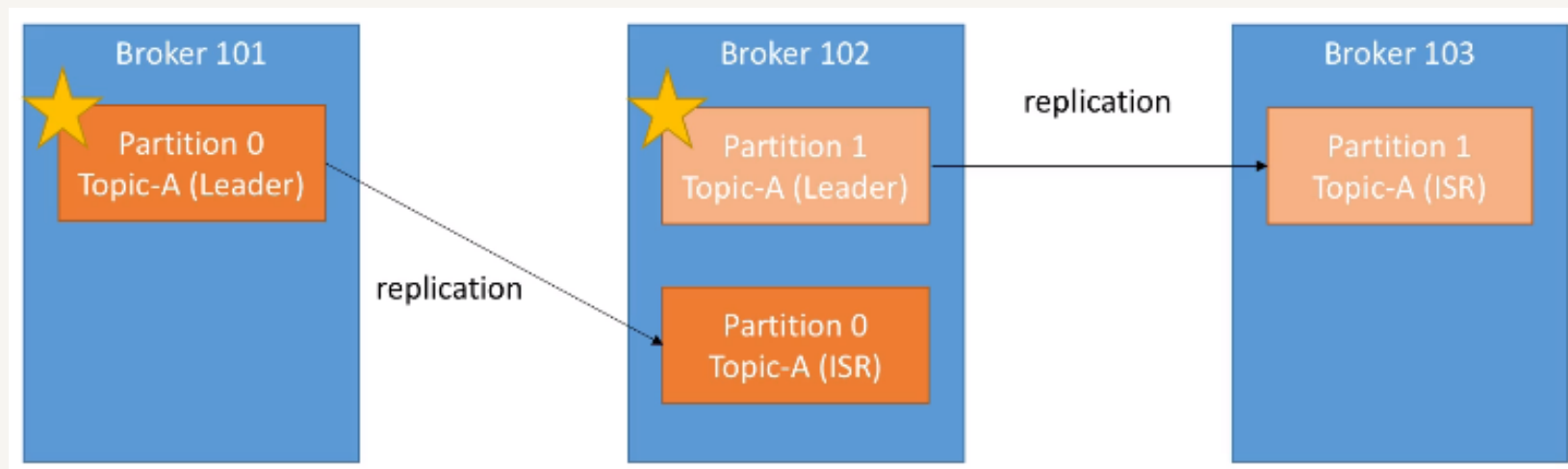
Errepikatze-faktorea — partizioak broker anitzetan

- PARTIZIO BAKOITZA REPLICATION-FACTOR BROKER-ETAN (0, 1, 2... broker-ID-ak)
- **LIDER BAKARRA** partizio-mailan — irakurketa-idazketa hortik bakarrik
- **Beste replicas FOLLOWER-AK** — sinkronizatzen daude leader-aren atzetik
- **FAULT-TOLERANCE**: N replication-factor → N-1 broker-en hutsegite jasan
- **REPLIKA-DISTRIBUZIOA AUTOMATIKOA** — Kafka-k broker ezberdinetan banatzen



— REPLIKAK BANAKETA

Partizioen erreplika-banaketa broker-en artean



- Topic bakoitzaren partizioak BROKER ANITZETAN errepikatuta (replication-factor>1)
- LEADER ETA FOLLOWER replikak banatuta broker-ezberdinetan
- **ISR (In-Sync Replicas)** — sinkronizatutako kopia-zerrenda (zaintza-kritikoa)
- Broker bat itzaltzean, bere LEADER-partizioek BESTE BROKER-ETAKO replikei pasatzen zaie
- **FAILOVER < 10 segundo lehenetsiz** — produkzio-mailako fault-tolerance

Leader replika eta ISR (In-Sync Replicas)

- **PARTIZIO BAKOITZAK LIDER-REPLIKA BAT du** — irakurketa eta idazketa guztiak hortik
- **FOLLOWER REPLIKAK** — datuak sinkronizatzen dituzte leader-tik (etengabe)
- **ISR (IN-SYNC REPLICA)** — sinkronizatuta dauden kopiak (leader barne)
- **ACKS=ALL + MIN.INSYNC.REPLICAS=2** — produkzio-estandarra fault-tolerance handian
- **BROKER ERORITZE-KASUAN** — ISR-tik replika bat automatikoki leader bilakatu (FAILOVER)

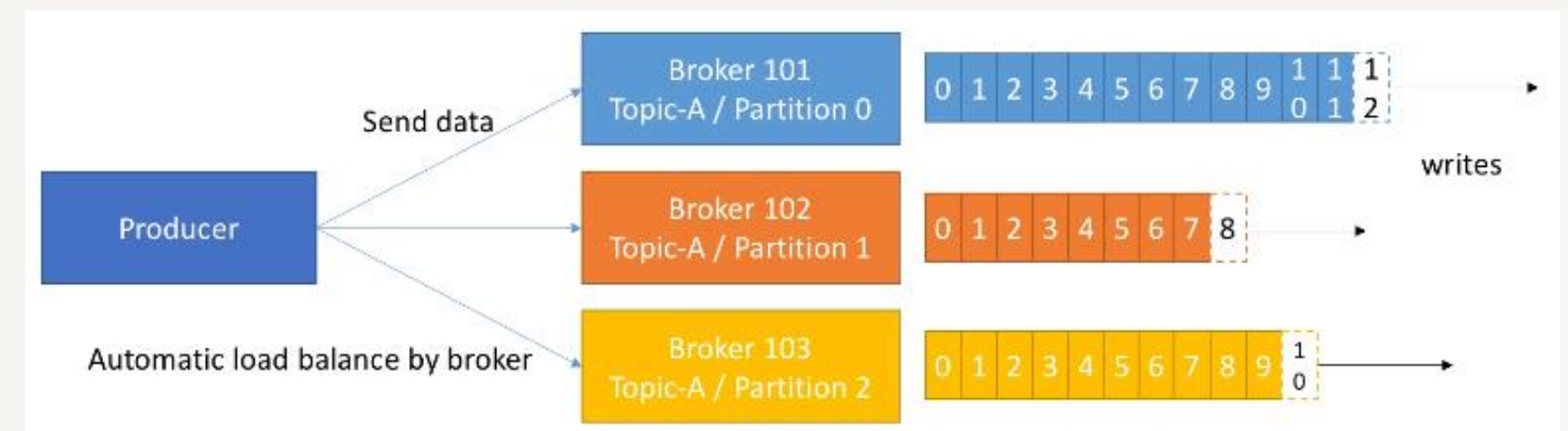
05

Ekoizleak (Producers)

— KARGA BANAKETA

Ekoizleak karga banatzen — round-robin vs gako

- **GAKORIK GABE** — Round-robin partition-banaketa (karga-balantzeatu uniforme)
- **GAKO-EKIN** — $\text{hash(gako)} \% \text{partition_count}$ (deterministikoa, gako berdina = partizio berdina)
- GAKOAK ORDENA bermatzen du entitate-mailan (banku-kontu, sentso-re-ID, etab.)
- EKOIZLEAK BROKER-LEADER-A AURKITZEN duen automatikoki metadatu-eskariekin
- **BROKER-ERORTZEAN** — berreskurapen automatikoa, client-aurrean transparente



ACK konfigurazioa — durabilitate-aukerak

- **ACKS=0** — Berrespenik EZ (fire-and-forget); datu-galera POSIBLEA, latencia <1ms
- **ACKS=1** — LIDER-aren berrespena; mugatu datu-galera, latencia 5-10ms (lehenetsia)
- **ACKS=ALL (-1)** — LIDER + erreplika guztiak (ISR-osoan); ZERO datu-galera, latencia 20-50ms
- **MEZU-GAKOA** — gako bera duten mezuak PARTIZIO berera; ordena entitate-mailan bermatua
- **BATCH-COMPRESSION** — gzip, snappy, lz4, zstd (producer-mailan konfiguragarriak)

Console producer mezu-gakoekin

```
●●● producer-gakoak.sh bash

1 # Console producer-a gakodun mezuekin
2 kafka-console-producer.sh --topic iabd-topic \
3   --property "parse.key=true" \
4   --property "key.separator=:" \
5   --bootstrap-server iabd-virtualbox:9092
6
7 # MEZU-ADIBIDEAK (gakoa:balioa formatuan):
8 # > gakoa1:balioa1
9 # > gakoa2:balioa2
10 # > gakoa1:balioa3
11 # > bezero_id:{"bezero_id":"1","izena":"Aitor","abizena":"Medrano"}
12
13 # OHARRAK:
14 #   parse.key=true      -> gako-balio bereizketa aktibatu
15 #   key.separator=:    -> : karakterea bereizle gisa
16 #   GAKO BERA DUTEN MEZUAK -> partizio berera doaz
17 #   Adib. 'gako1' beti partizio berean (1 eta 3 mezuak)
18 #
19 # Hartzaileak --property print.key=true ipini behar du gakoak ikusi
```

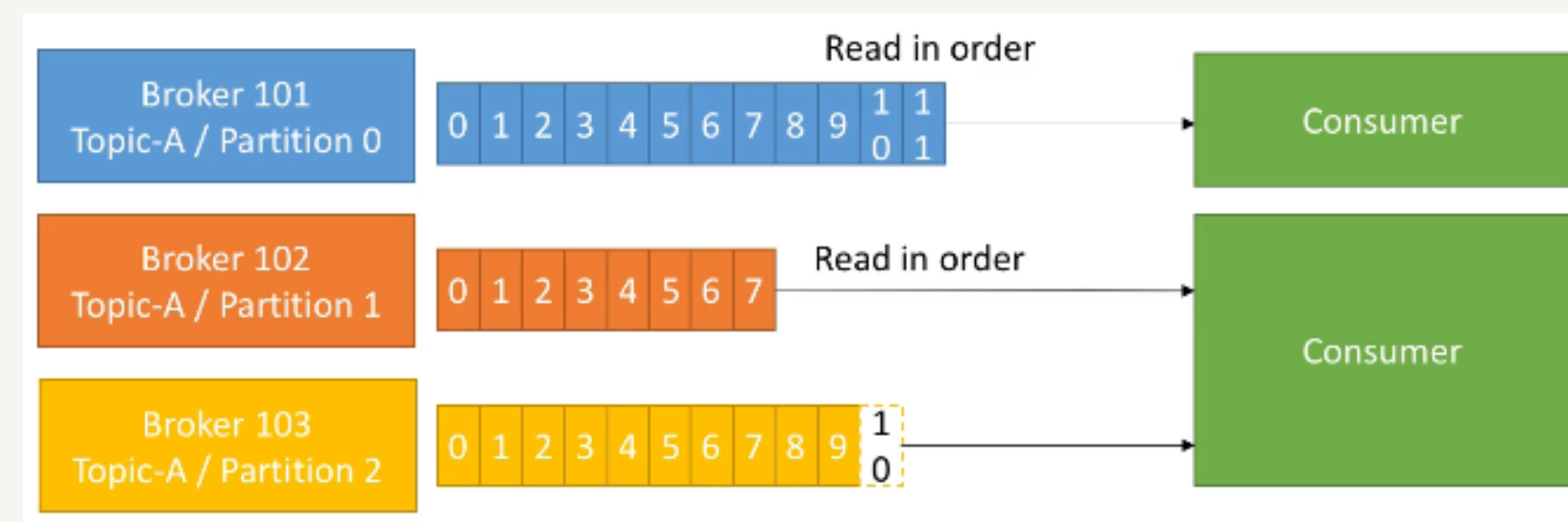
06

Kontsumitzaileak eta taldeak

— KONTSUMITZAILEAK

Kontsumitzaileak partizioetatik irakurtzen

- **Topic-PARTIZIO-etatik datuak irakurtzen** — partizio-bakoitzean ORDENAZ
- **Offset-jarraipenarekin** — non gelditu zen azken irakurketan
- N+1 offsetak EZIN IRAKURRI N amaitu arte (ordena-bermea)
- **POLL EREDUA** — consumer-ek bidaltzen ditu datu-eskariak (ez push)
- **MULTI-PARTITION CONSUMER** — partizio anitzetatik paraleloan irakur dezake



Kontsumitzaile-taldea (Consumer Group)

Consumer Group

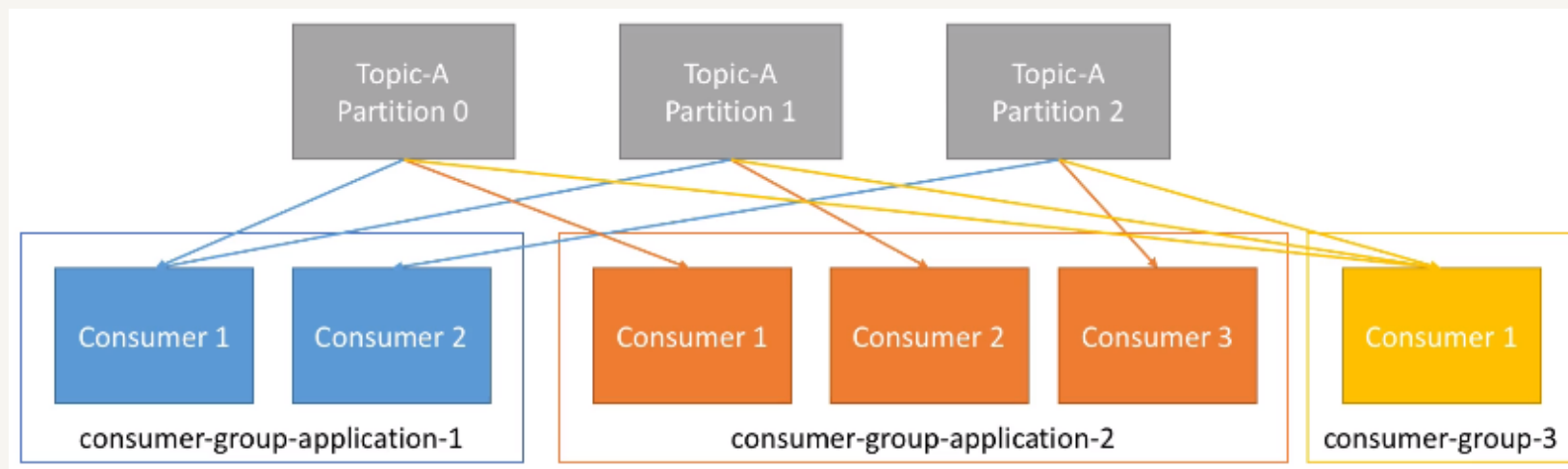
Talde bateko KONTSUMITZAILE BAKOITZAK topic baten PARTIZIOEN AZPIMULTZO bat jasotzen du. Talde DESBERDINAK datu BERDINAK jasotzen dituzte modu INDEPENDENTEAN. Taldeko kontsumitzaile kopurua partizioen kopurua baino HANDIAGOA bada, kontsumitzaile batzuk INAKTIBO geratzen dira.

PARALELISMOA

Talde berekoak = LANA BANATZEN dute (partition-banaketa). Talde ezberdinekoak = DENAK ENTZUTEN dute mezu BERDINAK.

— CONSUMER GROUP

Kontsumitzaile-taldea — partizio-banaketa



- TOPIC BAT N partziorekin (P0, P1, P2...)
- TALDE BAKAR BAT M kontsumitzailearekin (C1, C2...)
- $M \leq N$: kontsumitzaile bakoitzak partizio-azpimultzoa
- $M > N$: gehiegizko kontsumitzaileak INAKTIBO geratzen (idle, ez datuik)
- **Talde IZENA (group.id)** — paraleloan exekutatzen diren kontsumitzaileak elkartzeko

Talde anitzen adibidea — independentzia

- **Talde A** — 3 consumer, partizio-banaketa internoa
- **Talde B** — 2 consumer, partizio-banaketa internoa (beste konfigurazioa)
- **BI TALDEEK MEZU BERA irakurtzen** — bakoitzak BERE OFFSET-jarraipenarekin
- **Talde A consumer batek erortzean** — taldearen barneko rebalance soilik (B-k ez du sumatzen)
- Helburu-ezberdinak: A = analytics; B = audit-logging (independente prozesatuak)

```
iabd@iabd-virtualbox:/opt/kafka_2.13-2.8.1$ kafka
-console-consumer.sh --topic iabd-topic-group \
> --group iabd-app1 \
> --bootstrap-server iabd-virtualbox:9092
Prueba 1
Prueba 3
Prueba 8
Prueba 10
□

iabd@iabd-virtualbox:/opt/kafka_2.13-2.8.1$ kafka-console-consumer.sh --topic i
abd
-topic-group \
> --group iabd-app1 \
> --bootstrap-server iabd-virtualbox:
9092
Prueba 2
Prueba 4
Prueba 5
Prueba 6
Los mensajes se reparten entre los consum
idores del mismo grupo
Prueba 9
□

iabd@iabd-virtualbox:/opt/kafka_2.13-2.8.1$ kafka-console-producer.sh --top
ic
iabd-topic-group --bootstrap-server iab
d-virtualbox:9092
>[2022-02-22 17:10:22,631] WARN [Produc
er clientId=console-producer] Bootstrap
broker iabd-virtualbox:9092 (id: -1 ra
ck: null) disconnected (org.apache.kafk
a.clients.NetworkClient)
Prueba 1
>Prueba 2
>Prueba 3
>Prueba 4
>Prueba 5
>Prueba 6
>Los mensajes se reparten entre los con
sumidores del mismo grupo
>Prueba 8
>Prueba 9
>Prueba 10
```

Bi consumer talde berean — partizio-banaketa

... taldea-praktikan.sh

bash

```
1 # 1) 3 partiziozko topic-a sortu
2 kafka-topics.sh --create --topic iabd-topic-taldea \
3   --partitions 3 \
4   --bootstrap-server iabd-virtualbox:9092
5
6 # 2) TERMINAL 1 — Lehen kontsumitzailea 'iabd-app1' taldean
7 kafka-console-consumer.sh --topic iabd-topic-taldea \
8   --group iabd-app1 \
9   --bootstrap-server iabd-virtualbox:9092
10
11 # 3) TERMINAL 2 — Bigarren kontsumitzailea TALDE BEREAN
12 kafka-console-consumer.sh --topic iabd-topic-taldea \
13   --group iabd-app1 \
14   --bootstrap-server iabd-virtualbox:9092
15
16 # 4) TERMINAL 3 — Ekoizlea
17 kafka-console-producer.sh --topic iabd-topic-taldea \
18   --bootstrap-server iabd-virtualbox:9092
19
20 # EMAITZA: 3 partizio + 2 consumer
21 #   -> 1 consumer-ek 2 partizio, beste consumer-ak 1 partizio
22 #   -> mezuak BANATUTA agertuko dira terminal bietan
```

kafka-consumer-groups.sh — list eta describe

```
●●● kudeatu-taldeak.sh bash

1 # 1) TALDE GUZTIAK ZERRENDATU
2 kafka-consumer-groups.sh --list \
3   --bootstrap-server iabd-virtualbox:9092
4
5 # 2) TALDE ZEHATZA DESKRIBATU (egoera eta offset-ak)
6 kafka-consumer-groups.sh --describe --group iabd-app1 \
7   --bootstrap-server iabd-virtualbox:9092
8
9 # 3) OFFSET MANUALA RESET-egin (etenduran)
10 kafka-consumer-groups.sh --reset-offsets \
11   --group iabd-app1 --topic iabd-topic-taldea \
12   --to-earliest --execute \
13   --bootstrap-server iabd-virtualbox:9092
14
15 # OHARRAK:
16 #   --reset-offsets: consumer-talde GUZTIA gelditu behar
17 #                   (offsets aktiboak ez aldatzeko)
18 #   --to-earliest: hasieratik berriz irakurri
19 #   --to-latest: uneko bukaerara salto egin
20 #   --to-datetime: data zehatz batera
```

Talde-egoera — aktibo vs geldituta

●●● lag-emaiza.txt text

```
1 # AKTIBO (kontsumitzaileak dabilta, lag=0)
2 GROUP      TOPIC      PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
3 iabd-app1  iabd-topic-taldea 0      4              4              0
4 iabd-app1  iabd-topic-taldea 1      2              2              0
5 iabd-app1  iabd-topic-taldea 2      4              4              0
6
7 # GELDITUTA (5 mezu berri etorri dira)
8 GROUP      TOPIC      PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
9 iabd-app1  iabd-topic-taldea 2      4              5              1
10 iabd-app1  iabd-topic-taldea 1      2              4              2
11 iabd-app1  iabd-topic-taldea 0      4              6              2
12
13 # INTERPRETAZIOA:
14 #   AKTIBO: LAG=0 (consumer-ek mezu guztiak prozesatu dituzte)
15 #   GELDITUTA: LAG=1,2,2 -> 5 mezu pilatu dira partizioetan
16 #   BERRIRO ABIATUTAKOAN, lag jaisten hasiko da
```

— OFFSET PERSISTENTEA

Consumer offsets — azken irakurri-puntua



- **__CONSUMER_OFFSETS TOPIC BEREZIAN gordeta** — Kafka-k Kafka-bidez bere koordinazioa
- **ENABLE.AUTO.COMMIT (default true)** — periodikoki automatikoki commit (5 sec)
- **MANUAL COMMIT** — `consumer.commitSync()` edo `consumer.commitAsync()` — kontrol osoa
- **CHECKPOINT GISA** — consumer erortzean, azken commit-tik berriz hasten da
- **CONSUMER-TALDE-MAILAN** — ez consumer-indibidual-mailan (talde-rebalance-ondoren mantenitzen)

Entrega-semantikak — at-most/least/exactly once

- **AT MOST ONCE (gehienez behin)** — mezua jaso, offset-commit AURRETIK; akats-egonez gero MEZUA GALDU
- **AT LEAST ONCE (gutxienez behin)** — mezua prozesatu, offset-commit ONDOREN; akats-egonez gero BIKOIZTUAK
- **EXACTLY ONCE (zehazki behin)** — Kafka Streams + transakzio-API; consumer IDEMPOTENTEA behar
- **AT-LEAST-ONCE produkzio-ohikoena** — consumer-idempotency programatuz
- **EXACTLY-ONCE konplexua** — overhead handia, kasuetan ezinbestean (pago-prozesamendua, etab.)

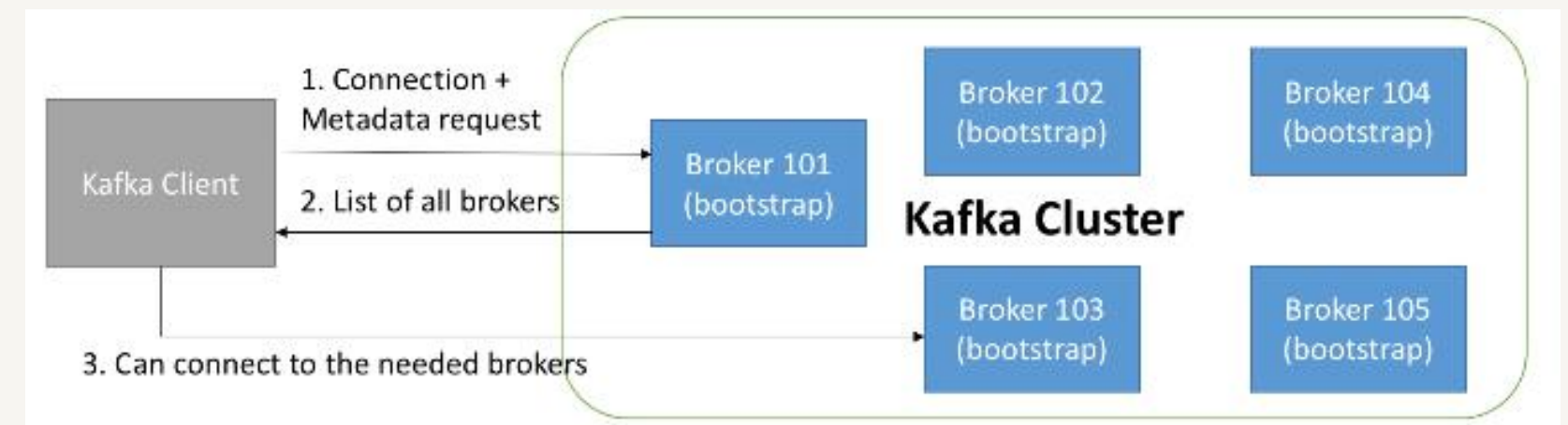
07

Broker Discovery

— BOOTSTRAP MEKANIKA

Edozein broker = entry-point kluster osora

- EDOZEIN BROKER 'bootstrap-server' gisa balio du
- **Lehen konexioan METADATU-ESKARIA** — kluster osoaren topologia jasotzen
- Erantzunean: broker-id-zerrenda + bakoitzaren host:port + partition-leader-mapping
- GERORA: client-ak ZUZENEAN konektatzen leader-broker-arekin dagokion partizioarentzat
- **BOOTSTRAP-LISTA OHIKOA** — b1:9092,b2:9092,b3:9092 (redundantzia konexio-faseko)



08 Zookeeper

— DEFINIZIOA

Zookeeper

Apache Zookeeper

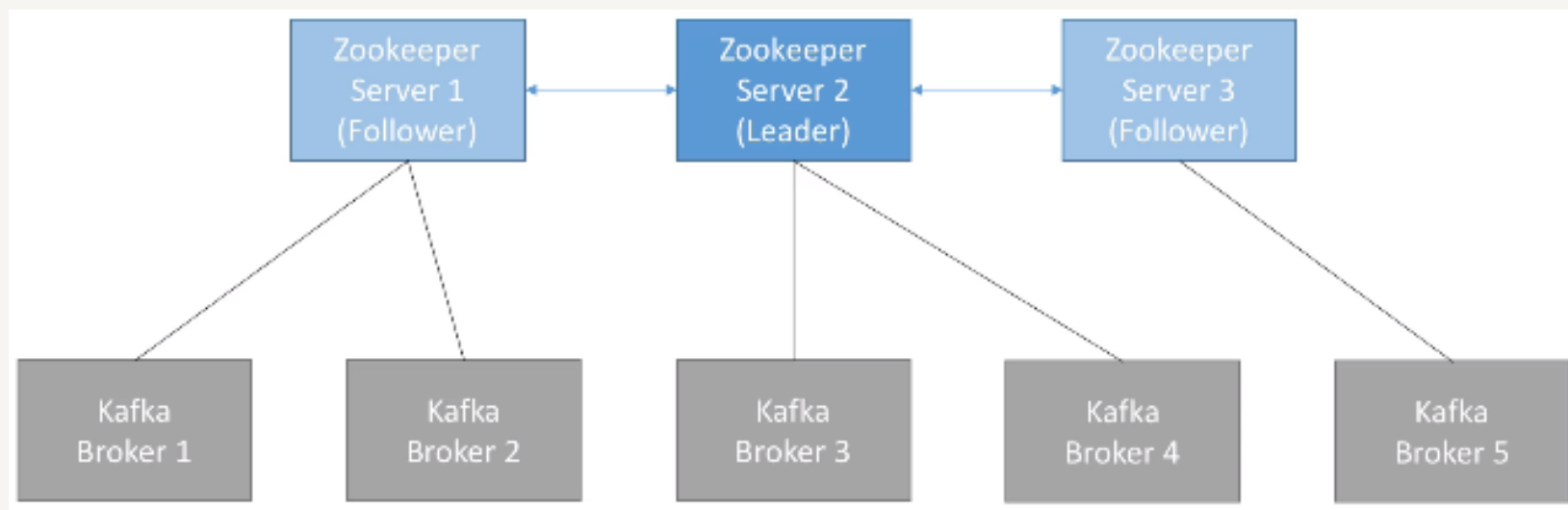
Kafka BANATURAKO konfigurazio, koordinazio eta hornidura mantentzen duen ZERBITZU KOORDINATZAILEA. Broker-rak kudeatzen ditu, partizioen LIDER-HAUTAKETAN laguntzen du eta aldaketen jakinarazpenak bidaltzen ditu (topic-sorkuntza, broker-en huts egitea/berreskuratzea).

KOORDINATZAILEA

Producer/Consumer-ek INOIZ EZ dute zuzenean Zookeeper-ekin komunikatzen — Broker-en bidez baino ez. Bakarrik infrastruktura-mailako koordinazio-rola.

— TOPOLOGIA

Kafka eta Zookeeper — kluster batean



- **ZOOKEEPER ENSEMBLE** — 3/5/7 nodo bakoiti (quorum-protokoloa)
- **KAFKA BROKERS** — Zookeeper-i registrer egiten (broker-membership)
- **CONTROLLER BROKER** — Zookeeper-bidez election (broker-batek rola hartzen)
- **PRODUCER/CONSUMER** — Broker-ekin BAKARRIK komunikatzen (Zookeeper-ekin inoiz EZ)
- **KRaft alternatiba (Kafka 4.0+ default)** — Zookeeper-rik gabe, broker-quorum-arekin

09

Kafka

bermeak

Kafka-ren bermeak — sistema-mailako kontratua

- MEZUAK PARTIZIO/TOPIC-EAN BIDALTZE-ORDENAN gehitzen dira (append-only log)
- KONTSUMITZAILEEK GORDEKETA-ORDENAN irakurtzen dituzte mezuak (partizio barruan)
- N ERREPIKATZE-FAKTOREAREKIN → N-1 broker-en hutsegiteak jasan daitezke gabe datu-galera
- **REPLICATION-FACTOR=3 PRODUKZIO-ESTANDAR** — mantentze-leiho + ustekabeko erortze aldi-berean
- **PARTIZIO-KOPURUA EZ ALDATZEN bitartean** — gako BERDINAK partizio BERDINA bermatua (ordena entitate-mailan)

10

1. Kasua — Kafka eta Python

kafka-python — instalazioa

... instalazioa.sh

```
1 # kafka-python liburutegia instalatu
2 pip install kafka-python
3
4 # Egiaztapena Python-en
5 python -c "import kafka; print(kafka.__version__)"
6 # Espero: 2.0.2 edo berriagoa
7
8 # OHARRAK:
9 # 1) Pure-Python inplementazioa (ez du C-konpilaketarik)
10 # 2) Produkzioan, confluent-kafka-python hobeagoa:
11 #    - C-binding (librdkafka azpian) — askoz azkarragoa
12 #    - Exactly-once-semantika onartzen
13 #    - Schema Registry integrazioa
14 #    pip install confluent-kafka
15 #
16 # 3) MongoDB-rako ariketa DF3.3-ean ere behar:
17 #    pip install pymongo Faker
```

bash

KafkaConsumer Python-en

●●● kafka_consumer.py

python

```
1 from kafka import KafkaConsumer
2 from json import loads
3
4 # Consumer-aren konfigurazioa
5 consumer = KafkaConsumer(
6     'iabd-topic',                # topic-izena
7     auto_offset_reset='earliest', # hasieratik irakurri
8     enable_auto_commit=True,      # offset-commit auto
9     group_id='iabd-taldea-1',     # talde-izena
10    value_deserializer=lambda m: loads(m.decode('utf-8')),
11    bootstrap_servers=['iabd-virtualbox:9092'])
12
13 # Mezuak etengabe jaso eta inprimatu
14 for m in consumer:
15     print(m.value)
16     # m.topic, m.partition, m.offset, m.key, m.value
17     # m.timestamp ere eskuragarri
```

KafkaConsumer — parametro nagusiak

- **LEHEN POSIZIONALA** — topic-izena (string) edo subscribe([...]) anitzentzat
- **BOOTSTRAP_SERVERS** — broker-zerrenda (list); redundantzia konexio-fasean
- **GROUP_ID** — kontsumitzaile-talde-izena; partition-assignment-aren oinarria
- **AUTO_OFFSET_RESET** — 'earliest' / 'latest' (hasiera-puntua group-offset-ik-ez denean)
- **ENABLE_AUTO_COMMIT** — True (default), background 5s; False = manual commit (kontrol-osoan)
- **VALUE_DESERIALIZER / KEY_DESERIALIZER** — bytes-tik objektu-rako (JSON, Avro, custom)

KafkaProducer Python-en

●●● kafka_producer.py

python

```
1 from kafka import KafkaProducer
2 from json import dumps
3 import time
4
5 # Producer-aren konfigurazioa
6 producer = KafkaProducer(
7     value_serializer=lambda m: dumps(m).encode('utf-8'),
8     bootstrap_servers=['iabd-virtualbox:9092'])
9
10 # 10 mezu bidali, segundo-tarteekin
11 for i in range(10):
12     mezua = {"izena": "ekoizlea " + str(i)}
13     producer.send("iabd-topic", value=mezua)
14     time.sleep(1)
15
16 # producer.flush() # buffer-aren mezuak bidaltzeko (ziurtatu)
17 # producer.close() # konexioak itxi
18
19 # IRTEERA consumer-ean (paraleloan exekutatuz):
20 # {'izena': 'ekoizlea 0'}
21 # {'izena': 'ekoizlea 1'}
22 # ...
23 # {'izena': 'ekoizlea 9'}
```

Gogoan hartu beharrekoak

-
- i* **Pub/Sub eredua** — producer/topic/consumer; ekoizleak eta kontsumitzaileak ELKARRENGANDIK ASKEAK, topic-bidez komunikatzen.
-
- ii* **Partizioak** = paralelismoaren UNITATEA; ordena PARTIZIO BARRUAN bermatua, GAKO-bidez entitate-mailan kontrolatua.
-
- iii* **Replication-factor** = $N \rightarrow N-1$ broker-erortze jasaten; produkzioan 3 estandarra (min-insync-replicas=2, acks=all).
-
- iv* **Consumer groups** — talde-barruan partition-banaketa (paralelizazioa); talde-anitz aldi berean (publish-once-consume-many).
-
- v* **LAG metrika** — produkzio-monitorizazio-kritikoa; consumer-osasunaren adierazlea.
-
- vi* **Kafka-Python (kafka-python)** — didaktiko; produkzioan confluent-kafka-python (C-binding, exactly-once, Schema Registry).
-

Galderak, zalantzak, iruzkinak.

APACHE KAFKA DOKUMENTAZIO OFIZIALA	kafka.apache.org/documentation
KAFKA CHEATSHEET (KOMANDOAK)	github.com/lensesio/kafka-cheat-sheet
CONFLUENT DEVELOPER (TUTORIALAK)	developer.confluent.io
JATORRIZKO MATERIALA	Aitor Medrano · CC BY-NC-SA